Network Fingerprint Generator (Network Profiler)

Course: Computer Networks

Project Overview:

The "Network Profiler" is a web-based application designed to capture, analyze, and visualize network traffic in real-time. The primary goal of this project is to allow users to input website URLs, capture their underlying network packets using Python (Scapy), and display statistical analysis such as packet sizes, total bytes, and traffic timelines on a React-based frontend. It provides deep insights into how different websites behave over the network.

Personal Understanding and Contribution:

Throughout this project, my main focus was on integrating the backend packet sniffing logic with the frontend visualization components. My specific contributions and learnings include:

- **Packet Extraction Logic:** Developed the logic in extract.py to process .pcap files. I implemented a time-series algorithm that groups network traffic into 1-second intervals (from 0s to 10s) to create an accurate flow of data over time, rather than relying on static or average metrics.
- **API Development:** Managed the Flask backend (app.py) to handle POST requests, ensuring seamless communication between the React frontend and the Python scripts. I corrected data transmission issues to ensure dynamic, real-time data is sent to the frontend instead of hardcoded dummy data.
- **Frontend Integration & State Management:** Worked on App.jsx to manage application states (loading, errors, single mode, comparison mode). I successfully integrated the Recharts library to visualize complex data.Made components based architecture like I divided charts, summary as a component and made a new file named as Summary.jsx, Charts.jsx, CompareCharts.jsx.
- **Understanding Network Protocols:** Gained hands-on experience in differentiating between TCP, UDP, HTTPS, and DNS traffic, and categorized websites (e.g., Streaming vs. Static Webpage) based on their packet behavior.

System Architecture & Tech Stack:

The project follows a modern Client-Server architecture:

- **Frontend:** React.js, Vite, HTML/CSS. Used Recharts for interactive graphs (Bar Charts and Line Charts).
- **Backend:** Python, Flask (RESTful API), Flask-CORS.
- **Network Tools:** Scapy for packet capturing and feature extraction.

Key Features & Visualizations:

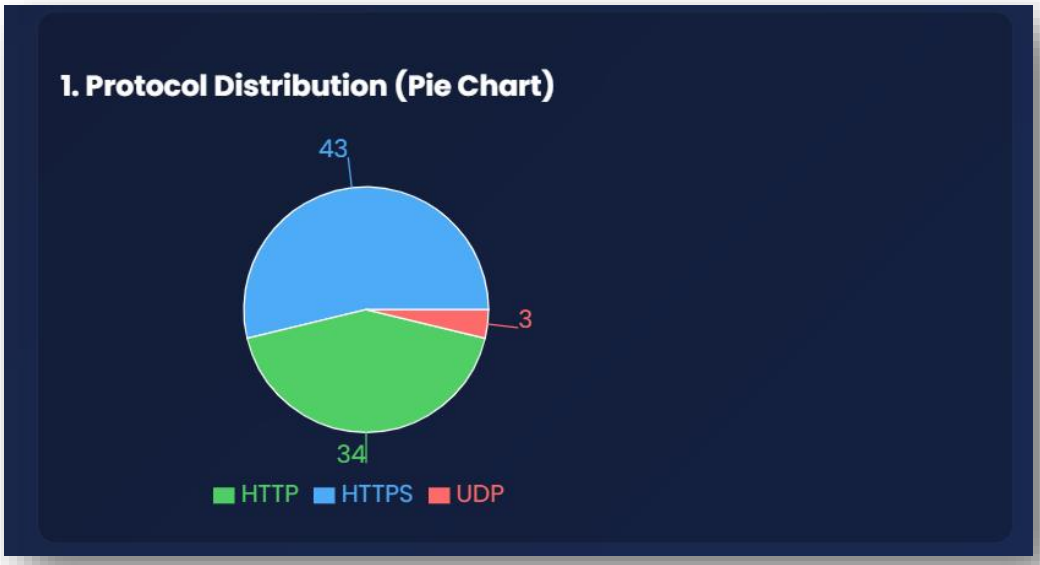
The application supports multiple analytical features and visual representations to make network data easy to understand:

A. Traffic Summary & Statistics At the top of the dashboard, the system provides a quick overview of the captured traffic.

- **Network Metrics:** Displays high-level metrics including Total Packets captured, Total Data Transferred (in bytes), Mean and Max Packet Sizes, and the number of Unique IP addresses interacted with. This gives a quick health check of the traffic volume.
- **Traffic Classification:** An algorithm automatically classifies the website's nature (e.g., "Streaming / Heavy Media", "API-Heavy", or "Static / Simple Webpage") based on the packet sizes and volume.

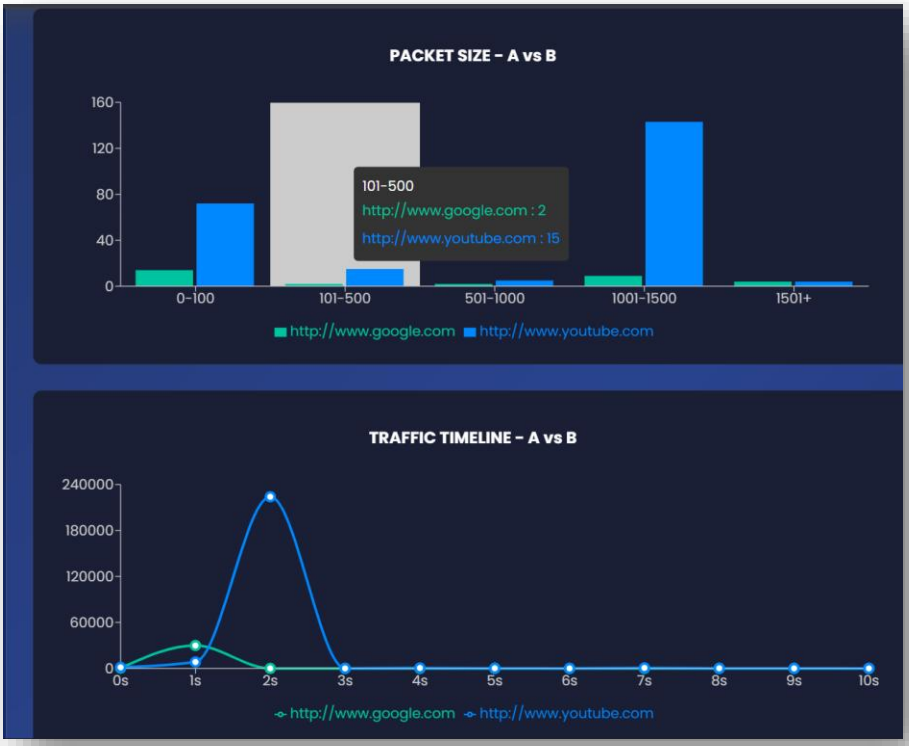
B. Protocol Distribution (Pie Chart)

- **Protocol Breakdown:** A visual Pie Chart representing the proportion of different network protocols active during the session.
- **Security & Traffic Insights:** It illustrates the percentage share of HTTPS, HTTP, TCP, UDP, and DNS traffic. This helps in quickly identifying whether the website relies on secure encrypted connections (HTTPS) or standard plain-text data (HTTP), and how many DNS lookups were required.



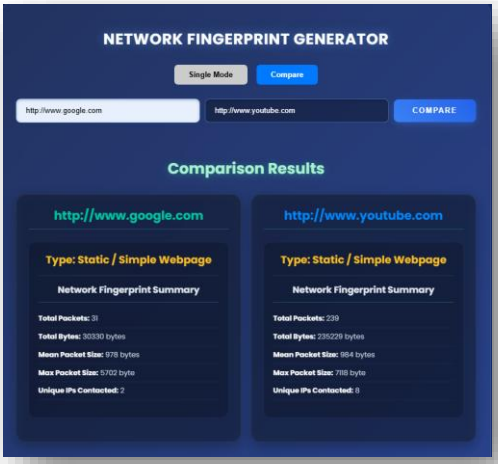
C. Traffic Visualizations (Comparison & Timelines)

1. **Packet Size Histogram (Bar Chart):** A grouped bar chart that compares the frequency of different packet sizes (e.g., 0-100 bytes, 1501+ bytes) between two websites. It helps in understanding if a site sends many small packets (like APIs) or fewer large packets (like media streaming).
2. **Traffic Timeline (Wavy Line Chart):** A multi-line chart representing the continuous flow of data (in bytes) over a 10-second window. It shows the real-time spikes and drops in network activity for the targeted URLs.



D. Comparison Mode

- **Side-by-Side Analysis:** This mode allows users to input two different URLs simultaneously. The system generates comparative summary cards, histograms, and merged timelines, allowing a direct comparison of their network footprints.



Conclusion

This project bridged the gap between theoretical computer networks and practical web development. By building the Network Profiler, I successfully learned how to capture live packets, extract meaningful features, build robust APIs, and render dynamic data on a modern frontend interface.

The complete source code of this project, including the backend scripts, frontend components, and this technical report, has been uploaded to GitHub for transparency and evaluation

GitHub Repository Link: https://github.com/the-shahzaib-asif/Network-Profiler